



MAPA – Material de Avaliação Prática da Aprendizagem

Nome: Kleber Alves Pereira Junior	R.A 24118233-5
Curso: Análise e Desenvolvimento de Sistemas	
Disciplina: PROGRAMAÇÃO AVANÇADA	

Instruções para Realização da Atividade

1. Revise seu arquivo antes do envio. Certifique-se de que é o arquivo correto, formato correto, se contempla todas as demandas da atividade etc.
2. Após o envio não serão permitidas alterações.
3. Durante a disciplina, procure sanar suas dúvidas pontuais em relação ao conteúdo relacionado à atividade. Porém, não são permitidas correções parciais, ou seja, enviar para que o professor possa fazer uma avaliação prévia e retornar para que o aluno possa ajustar e enviar novamente. Isso não é permitido, pois descaracteriza o processo de avaliação.
4. Ao utilizar quaisquer materiais de pesquisa referencie conforme as normas da ABNT;

Em caso de dúvidas, entre em contato com seu Professor Mediador.

Bons estudos!



AGORA, É COM VOCÊ!

A Programação Orientada a Objetos (POO) é um dos paradigmas mais utilizados no desenvolvimento de software, permitindo a criação de sistemas modulares, reutilizáveis e de fácil manutenção. Em Java, a Programação Orientada a Objetos é baseada em classes, objetos, encapsulamento, herança e polimorfismo, conceitos essenciais para a modelagem de soluções eficientes e escaláveis.

Neste conjunto de atividades, os estudantes praticarão e consolidarão esses fundamentos, avançando progressivamente desde a criação de classes e objetos até a implementação de herança, polimorfismo e interfaces. O objetivo é não apenas compreender a teoria, mas também aplicar os conceitos em situações reais, desenvolvendo habilidades que são fundamentais para o mercado de trabalho.

Referência: OLIVEIRA JUNIOR, E. A.; PEREIRA, R. de L. **Programação Avançada**. Maringá: UniCesumar, 2016.

Classes e Objetos

1 Criando a primeira classe

Crie uma classe chamada Pessoa com os seguintes atributos:

nome (String)

idade (int)

cidade (String)

No método main, instancie um objeto da classe Pessoa e exiba os dados no console.

Métodos e Construtores

2 Classe "Retângulo" com métodos

Crie uma classe Retângulo com os atributos largura e altura e um método:

calcularArea() → Retorna a área do retângulo.

No main, instancie um Retângulo, defina os valores e exiba a área.

3 Conta Bancária (Construtor e Métodos)

Crie a classe ContaBancaria com os atributos:

titular

saldo

Implemente os métodos:

depositar(double valor)

sacar(double valor)

exibirSaldo()

No main, crie um objeto e teste as operações de saque e depósito.

Encapsulamento (Getters e Setters)

4 Classe "Produto" (Encapsulamento)

Crie uma classe Produto com os atributos privados:

nome (String)

preco (double)

Implemente os métodos **getters e setters** para acessar e modificar os atributos.



No main, instancie um objeto Produto, defina os valores e exiba as informações.

Herança e Polimorfismo

5 Criando a classe "Funcionário"

Crie uma classe Funcionário com os atributos:

nome

salario

Crie o método calcularBonus(), que retorna **10% do salário**.

Agora, crie a classe Gerente, que **herda** de Funcionário, e **sobrescreva** calcularBonus() para retornar **20% do salário**.

Teste o polimorfismo no main:

Crie um Funcionário e um Gerente, e exiba o bônus de cada um.

6 Sistema de Veículos (Herança e Polimorfismo)

Crie uma **classe base** chamada Veículo com um método mover().

Agora, crie duas classes-filhas:

Carro → Sobrescreva mover() para exibir **"O carro está se movendo rapidamente"**.

Bicicleta → Sobrescreva mover() para exibir **"A bicicleta está se movendo lentamente"**.

No main, teste o polimorfismo chamando mover() em objetos Carro e Bicicleta.

Como entregar a atividade:

A atividade deverá ser produzida em um arquivo do tipo TEXTO, conforme TEMPLATE anexado no MATERIAL DA DISCIPLINA, disponibilizado no Studeo, e DEVE ser entregue com a extensão (.PDF). Depois, deve ser anexado no ambiente da Atividade no STUDEO.

Obs.: inserir o print com o código e a execução do projeto.

Dicas para realizar a atividade:

1. Durante as aulas, o professor fornecerá dicas que podem ser utilizadas para a confecção das suas atividades. Assim, é de suma importância participar das aulas ao vivo ou assisti-las posteriormente.
2. Assista às aulas conceituais da disciplina.

Orientações:

Plágios e cópias indevidas serão penalizadas com descontos na nota, podendo chegar a zero.

Não são permitidas correções parciais no decorrer do módulo, pois a interpretação da atividade também faz parte da avaliação.

Atenção ao prazo de entrega da atividade. Sugerimos que envie sua atividade antes do prazo final para evitar transtornos e lentidão nos servidores. Evite o envio de atividade em cima do prazo.

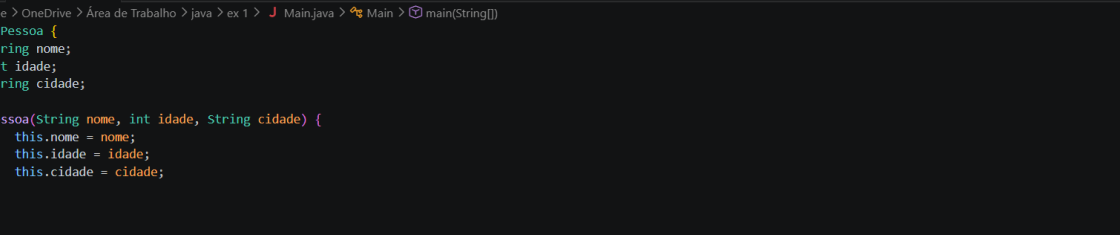
Boa atividade!

Coloque sua resposta a continuação:

Print do código e execução:

Resposta 1 – Classe Pessoa

{Inserir aqui o print da IDE com o código e a execução funcionando}

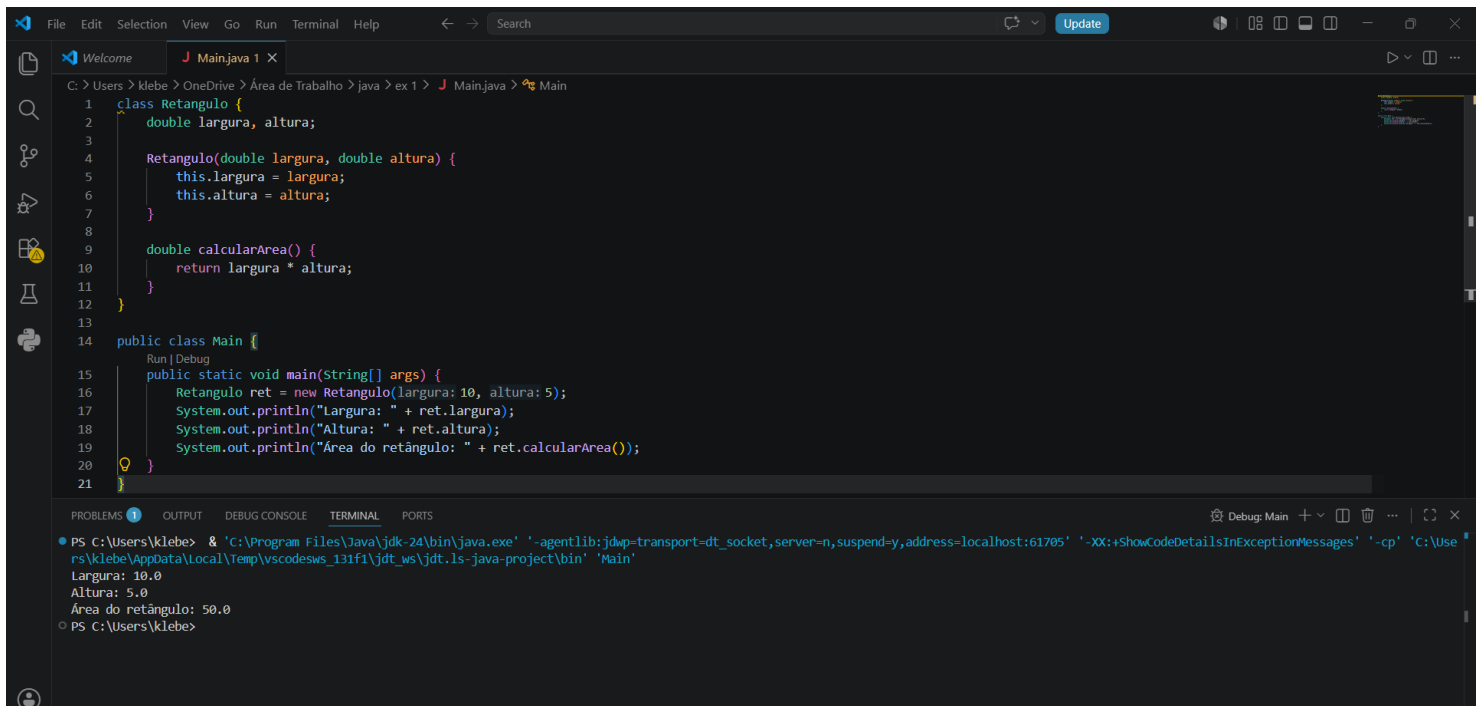


```
File Edit Selection View Go Run Terminal Help
Welcome J Main.java 1 X
C:\> Users > klebe > OneDrive > Área de Trabalho > java > ex 1 > J Main.java > Main > main(String[])
1 class Pessoa {
2     String nome;
3     int idade;
4     String cidade;
5
6     Pessoa(String nome, int idade, String cidade) {
7         this.nome = nome;
8         this.idade = idade;
9         this.cidade = cidade;
10    }
11 }
12
13 public class Main {
14     Run | Debug
15     public static void main(String[] args) {
16         Pessoa pessoa1 = new Pessoa(nome: "Kleber", idade: 21, cidade: "Imperatriz");
17         System.out.println("Nome: " + pessoa1.nome);
18         System.out.println("Idade: " + pessoa1.idade);
19         System.out.println("Cidade: " + pessoa1.cidade);
20     }
21 }
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Klebe> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:57217' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Use
rs\Klebe\AppData\Local\Temp\vscode\ws_131f1\jdt_ws\jdt.ls-java-project\bin' 'Main'
Nome: Kleber
Idade: 21
Cidade: Imperatriz
PS C:\Users\Klebe>
```



Resposta 2 – Classe Retângulo

{Inserir aqui o código da classe Retângulo e o método calcularArea(), com o print da execução}



```
1 class Retangulo {
2     double largura, altura;
3
4     Retangulo(double largura, double altura) {
5         this.largura = largura;
6         this.altura = altura;
7     }
8
9     double calcularArea() {
10        return largura * altura;
11    }
12 }
13
14 public class Main {
15     Run | Debug
16     public static void main(String[] args) {
17         Retangulo ret = new Retangulo(largura: 10, altura: 5);
18         System.out.println("Largura: " + ret.largura);
19         System.out.println("Altura: " + ret.altura);
20         System.out.println("Área do retângulo: " + ret.calcularArea());
21     }
22 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

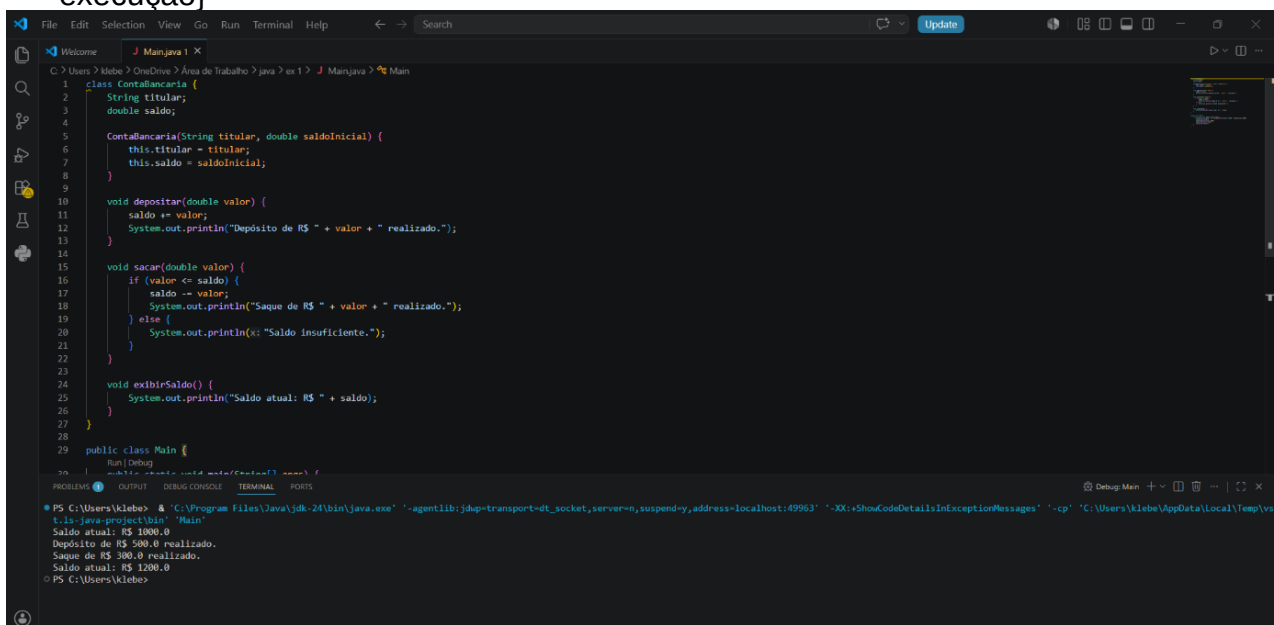
PS C:\Users\klebe> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:61705' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\klebe\AppData\Local\Temp\vscodesws_131f1\jdt_ws\jdt.ls-java-project\bin' 'Main'

Largura: 10.0
Altura: 5.0
Área do retângulo: 50.0

PS C:\Users\klebe>

Resposta 3 – Conta Bancária

{Inserir aqui o código com os métodos depositar, sacar, exibirSaldo e o print da execução}



```
1 class ContaBancaria {
2     String titular;
3     double saldo;
4
5     ContaBancaria(String titular, double saldoInicial) {
6         this.titular = titular;
7         this.saldo = saldoInicial;
8     }
9
10    void depositar(double valor) {
11        saldo += valor;
12        System.out.println("Depósito de R$ " + valor + " realizado.");
13    }
14
15    void sacar(double valor) {
16        if (valor <= saldo) {
17            saldo -= valor;
18            System.out.println("Saque de R$ " + valor + " realizado.");
19        } else {
20            System.out.println("Saldo insuficiente.");
21        }
22    }
23
24    void exibirSaldo() {
25        System.out.println("Saldo atual: R$ " + saldo);
26    }
27 }
28
29 public class Main {
30     Run | Debug
31     public static void main(String[] args) {
32         ContaBancaria conta = new ContaBancaria("João", 1000.0);
33         conta.exibirSaldo();
34         conta.depositar(500.0);
35         conta.exibirSaldo();
36         conta.sacar(300.0);
37         conta.exibirSaldo();
38     }
39 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\klebe> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49963' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\klebe\AppData\Local\Temp\vscodesws_131f1\jdt_ws\jdt.ls-java-project\bin' 'Main'

Saldo atual: R\$ 1000.0
Depósito de R\$ 500.0 realizado.
Saque de R\$ 300.0 realizado.
Saldo atual: R\$ 1200.0

PS C:\Users\klebe>



```
File Edit Selection View Go Run Terminal Help
C:\Users\klebe> OneDrive > Área de Trabalho > java > ex1 > J Mainjava > Main

1 class ContaBancaria {
10     void depositar(double valor) {
11         System.out.println("Depósito de R$ " + valor + " realizado.");
12     }
13 }
14
15 void sacar(double valor) {
16     if (valor <= saldo) {
17         saldo -= valor;
18         System.out.println("Saque de R$ " + valor + " realizado.");
19     } else {
20         System.out.println(x: "Saldo insuficiente.");
21     }
22 }
23
24 void exibirSaldo() {
25     System.out.println("Saldo atual: R$ " + saldo);
26 }
27 }
28
29 public class Main {
30     public static void main(String[] args) {
31         ContaBancaria conta = new ContaBancaria(titular: "João", saldoInicial: 1000);
32         conta.exibirSaldo();
33         conta.depositar(valor: 500);
34         conta.sacar(valor: 300);
35         conta.exibirSaldo();
36     }
37 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\klebe> & "C:\Program Files\Java\jdk-24\bin\java.exe" "-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:49963" "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\klebe\AppData\Local\Temp\vscode-15-java-project\bin" "Main"
Saldo atual: R$ 1000.0
Depósito de R$ 500.0 realizado.
Saque de R$ 300.0 realizado.
Saldo atual: R$ 1200.0
PS C:\Users\klebe>
```

Resposta 4 – Classe Produto com Encapsulamento

{Inserir código com getters e setters e exemplo de uso no main + print da execução}

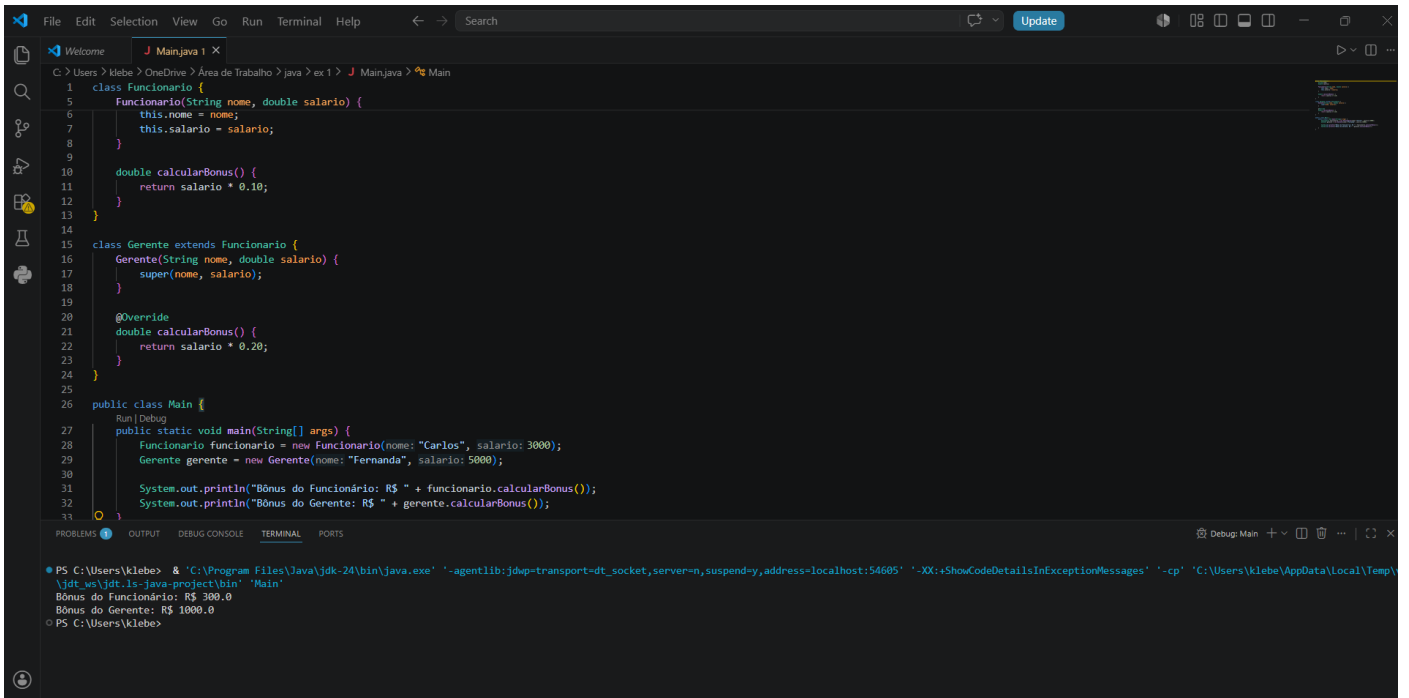
```
File Edit Selection View Go Run Terminal Help
C:\Users\klebe> OneDrive > Área de Trabalho > java > ex1 > J Mainjava > Main

1 class Produto {
2     private String nome;
3     private double preco;
4
5     Produto(String nome, double preco) {
6         this.nome = nome;
7         this.preco = preco;
8     }
9
10    public String getNome() { return nome; }
11    public double getPreco() { return preco; }
12 }
13
14 public class Main {
15     public static void main(String[] args) {
16         Produto produto = new Produto(nome: "Notebook Gamer", preco: 4500.00);
17         System.out.println("Produto: " + produto.getNome());
18         System.out.println("Preço: R$ " + produto.getPreco());
19     }
20 }
```



Resposta 5 – Funcionário e Gerente (Herança e Polimorfismo)

{Inserir código das classes com sobrescrita do método calcularBonus e print da execução}



```
File Edit Selection View Go Run Terminal Help
C:\Users\klebe> OneDrive > Área de Trabalho > java > ex1 > J Main.java > Main

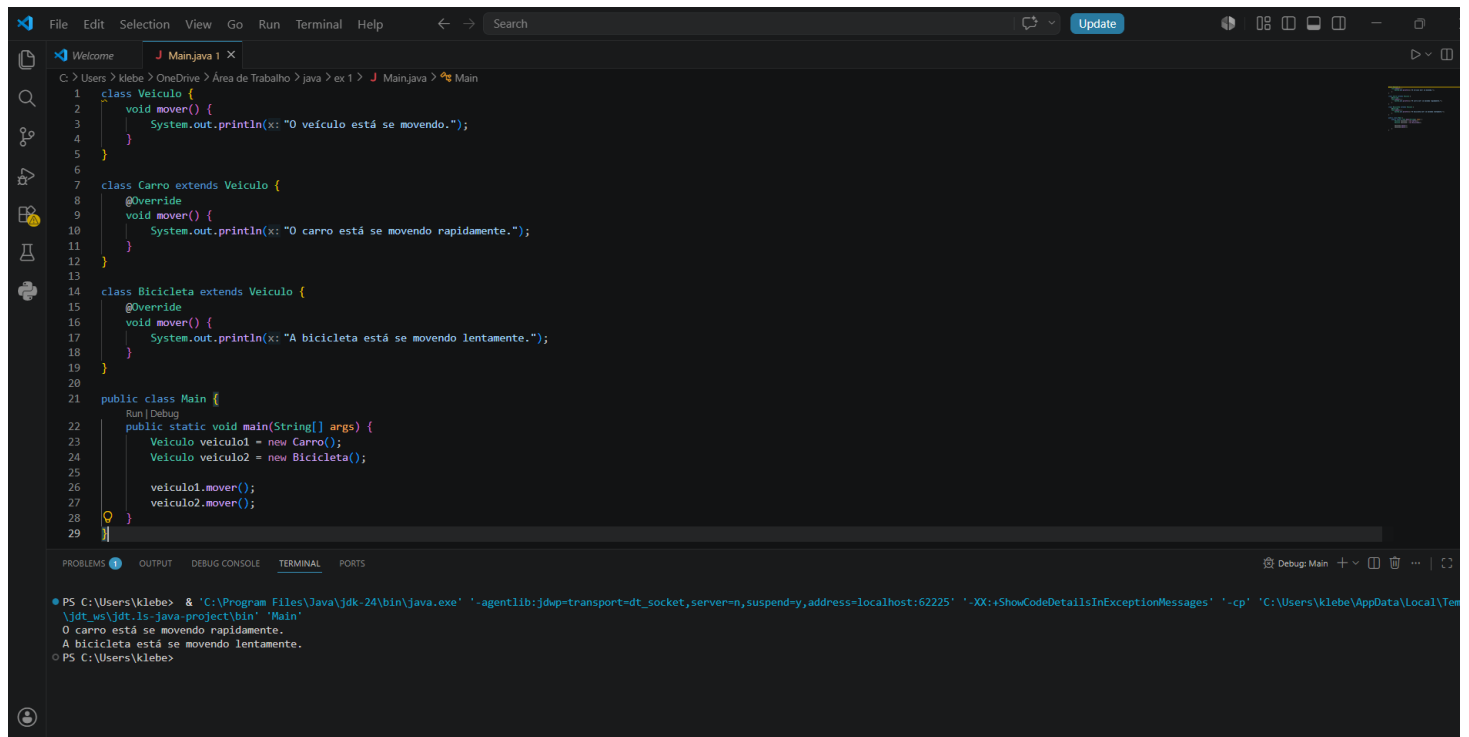
1 class Funcionario {
2     Funcionario(String nome, double salario) {
3         this.nome = nome;
4         this.salario = salario;
5     }
6
7     double calcularBonus() {
8         return salario * 0.10;
9     }
10 }
11
12 class Gerente extends Funcionario {
13     Gerente(String nome, double salario) {
14         super(nome, salario);
15     }
16
17     @Override
18     double calcularBonus() {
19         return salario * 0.20;
20     }
21 }
22
23 public class Main {
24     public static void main(String[] args) {
25         Funcionario funcionario = new Funcionario("Carlos", 3000);
26         Gerente gerente = new Gerente("Fernanda", 5000);
27
28         System.out.println("Bônus do Funcionário: R$ " + funcionario.calcularBonus());
29         System.out.println("Bônus do Gerente: R$ " + gerente.calcularBonus());
30     }
31 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\klebe> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:54605' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\klebe\AppData\Local\Temp\jdt_ws\jdt.ls-java-project\bin' 'Main'
Bônus do Funcionário: R$ 300.0
Bônus do Gerente: R$ 1000.0
PS C:\Users\klebe>
```



Resposta 6 – Sistema de Veículos (Herança e Polimorfismo)

{Inserir código das classes Veículo, Carro e Bicicleta + print da execução de cada}



```
1 class Veiculo {
2     void mover() {
3         System.out.println(x: "O veículo está se movendo.");
4     }
5 }
6
7 class Carro extends Veiculo {
8     @Override
9     void mover() {
10         System.out.println(x: "O carro está se movendo rapidamente.");
11     }
12 }
13
14 class Bicicleta extends Veiculo {
15     @Override
16     void mover() {
17         System.out.println(x: "A bicicleta está se movendo lentamente.");
18     }
19 }
20
21 public class Main {
22     Run | Debug
23     public static void main(String[] args) {
24         Veiculo veiculo1 = new Carro();
25         Veiculo veiculo2 = new Bicicleta();
26
27         veiculo1.mover();
28         veiculo2.mover();
29     }
30 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\klebe> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:62225' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\klebe\AppData\Local\Temp\jdt_ws\jdt_ws-java-project\bin' 'Main'

O carro está se movendo rapidamente.

A bicicleta está se movendo lentamente.

PS C:\Users\klebe>

Importante:

- Certifique-se de que todos os prints estejam legíveis e mostrem a execução correta do projeto.
- O arquivo final deve ser exportado em formato **.PDF**, conforme orientações da disciplina.